COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithm	**SECTION:**	CPE12S4
DATE PERFORMED:	October 16, 2026	**DATE SUBMITTED:**	October 21, 2026
NAME:	Cardenas, Lea Marie De Vera, Sophia	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY 12.1 ALGORITHMIC STRATEGIES

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

STRATEGY	ALGORITHM	ANALYSIS
Recursion	- Factorial, recursive tree traversal and Fibonacci. - ***Tree Traversals*** specifically the in-order, pre-order, and post-order traversals.	- In this algorithm technique, it is involved in calling out the function to solve the problem by breaking it down into smaller subsproblems. The base case for this is crucial to stop the recursion process, preventing the infinite loops. The worst case occurs when the recursion calls continue indefinitely, leading to inefficient solutions. - All of our given recursion algorithms, like the tree traversal, uses recursion strategy since the algorithm recursively visits the left and right subtrees, depending on the traversal order.
Brute Force	- Examples of the **Brute Force** algorithm from our previous HOAs is the ***Linear Searching***. - Linear searching is a straightforward algorithm that checks each and every element thereof. - Another is the ***Bubble Sort***, where it goes in each element until the end of the array to	- This is a straightforward approach that systematically enumerates all possible solutions and checks whether each satisfies the problem statement and it prioritizes simplicity and correctness over speed. - In this algorithm technique, the program checks each element in a sequential

	find its way to guarantee the array is sorted.	manner until the proposed target element is found from the array list. It does not skip any element from the array list.
Backtracking	- ***Tree Traversals*** can be considered under the backtracking strategy since it can return from its previous root and explore its adjacent nodes/roots. - Although the tree traversals technique falls under this strategy, we think that the tree traversal is more fitting in the recursion than backtracking.	- This algorithm technique, solving a problem recursively by trying to build a solution incrementally, like one piece at a time, and it removes any candidate that fails to satisfy the constraints at any point. - This specific type of strategy, from the word itself, backtrack, once it reaches an end, it returns to an earlier point to explore another branch. This matches backtracking only if the implementation involves exploring alternatives and reversing choices.
Greedy	- ***Selection Sort*** is the nearest example we could come up with since it selects the smallest available element **index i** and **index j** and the **minIndex().** - It is relatively close to ***greedy strategy*** but it has similarities to a standard greedy algorithm strategy.	- This algorithm technique is a global optimal solution that can be achieved by making locally optimal choices that choose what looks the best at the current moment without regard for future consequences and it will lead to the best result. - This type of algorithm strategy refers to repeatedly selecting the smallest element from the unsorted portion and placing it in its correct position.
Divide-and-Conquer	- The main examples of this strategy is the ***Merge Sort*** and ***Quick Sort***. - We remember how we group and divide them into smaller arrays, then proceed to arrange them accordingly.	- This algorithm technique is a multi-step that breaks down a complex problem into smaller subproblems, solves each subproblem, and lastly it merge the results to form the complete solution. - The Divide-and-Conquer strategy selects pivot, which is

		basically used in ***Quick Sorting*** technique, since it divides and makes partitions to make smaller groups in which it sorts those groups, and produces a sorted array.

Table 12-2. Memoization Implementation

SCREENSHOTS

```
#include <iostream>
#include <algorithm>

int getMinStepsMemo(int n, int memo[]){
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];
    int r = 1 + getMinStepsMemo(n-1, memo);
    if (n%2 == 0) r = std::min(r, 1 + getMinStepsMemo(n/2, memo));
    if (n%3 == 0) r = std::min(r, 1 + getMinStepsMemo(n/3, memo));
    memo[n] = r;
    return r;
}

int main(){
    int memoArr[] = {1, 4, 7, 10};

    // MEMOIZATION
    std::cout << "Memoization\n";
    for (int t : memoArr){
        for(int i = 0; i <= t; i++)
            memo[i] = -1;
        std::cout << "n = " << t << "  ->  minimum steps = " << getMinStepsMemo(t, memo) << '\n';
    }
```

```
#include <iostream>
#include <algorithm>
#include "Memoization.h"
#include "Bottom_up.h"
#include "Count_paths.h"

int memo[1001];

int main(){
    int memoArr[] = {1, 4, 7, 10};

    // MEMOIZATION
    std::cout << "Memoization\n";
    for (int t : memoArr){
        for(int i = 0; i <= t; i++)
            memo[i] = -1;
        std::cout << "n = " << t << "  ->  minimum steps = " << getMinStepsMemo(t, memo) << '\
    }
```

	Memoization n = 1 -> minimum steps = 0 n = 4 -> minimum steps = 2 n = 7 -> minimum steps = 3 n = 10 -> minimum steps = 3
OBSERVATION ANALYSIS	***It shows here that the memoization helps to find the minimum number of steps like here in our outputs, efficiently storing previous calculated numbers. As the value of n increases, the minimum steps also increase step by step. So this approach avoids repeated calculations and improves our codings.***

Table 12-3. Bottom-Up Dynamic Programming Implementation

SCREENSHOTS

```
#include <iostream>
#include <algorithm>

int getMinStepsBottomUp(int n, int memo[]){
    int dp[n+1], i;
    dp[1] = 0;
    for( i = 2 ; i <= n ; i ++ ){
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = std::min(dp[i], 1+ dp[i/2]);
        if(i%3==0) dp[i] = std::min(dp[i], 1+ dp[i/3]);
    }
    return dp[n];
}

// BOTTOM UP
std::cout << "\nBottom Up Dp\n";
for (int t : memoArr){
    std::cout << "n = " << t << "  ->  minimum steps = " << getMinStepsBottomUp(t, memo) << '\n';
}
```

```
#include <iostream>
#include <algorithm>
#include "Memoization.h"
#include "Bottom_up.h"
#include "Count_paths.h"

int memo[1001];

int main(){
    int memoArr[] = {1, 4, 7, 10};

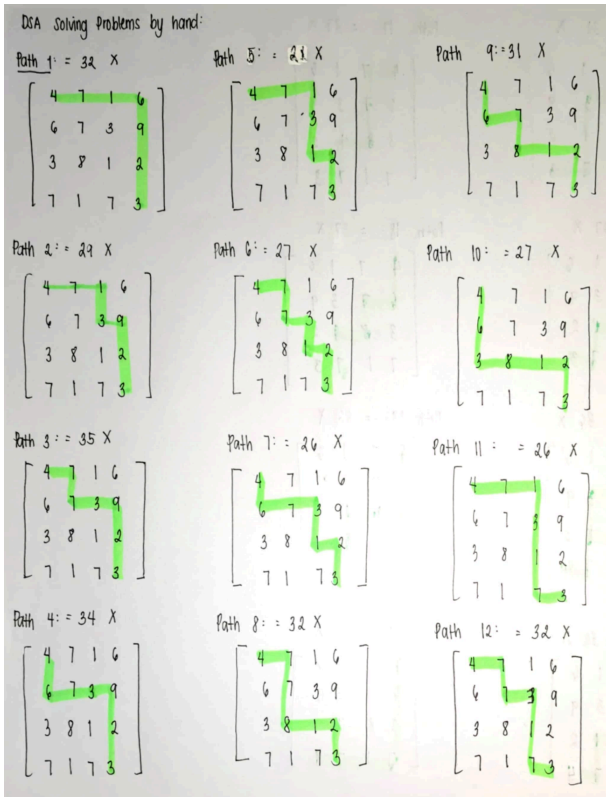
    // MEMOIZATION
    std::cout << "Memoization\n";
    for (int t : memoArr){
        for(int i = 0; i <= t; i++)
            memo[i] = -1;
        std::cout << "n = " << t << "  ->  minimum steps = " << getMinStepsMemo(t, memo) << '\n';
    }

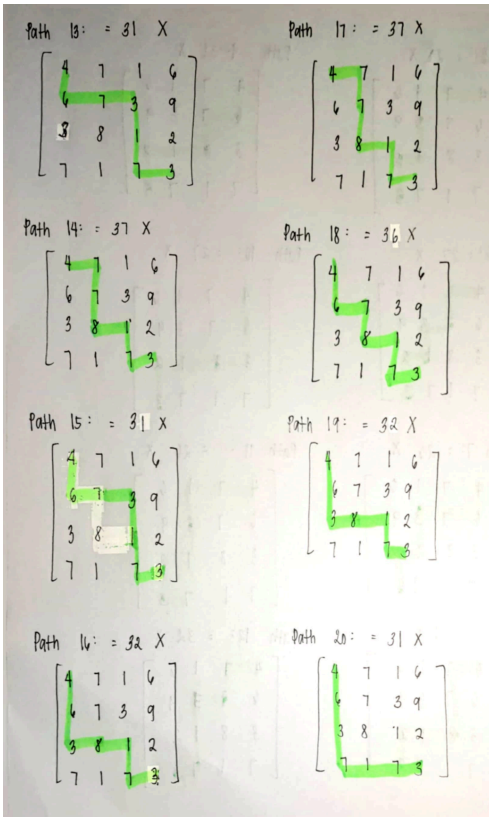
    // BOTTOM UP
    std::cout << "\nBottom Up Dp\n";
    for (int t : memoArr){
        std::cout << "n = " << t << "  ->  minimum steps = " << getMinStepsBottomUp(t, memo) << '\n';
    }
```

	Bottom Up Dp n = 1 -> minimum steps = 0 n = 4 -> minimum steps = 2 n = 7 -> minimum steps = 3 n = 10 -> minimum steps = 3
OBSERVATION ANALYSIS	***We observed that the Bottom-Up DP approach produces the correct minimum steps for every tested value of n. As n increases, the number of steps does not always increase because the program finds the most efficient sequence of steps.***

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies
ILO C: Create C++ code to solve problems using learned algorithmic strategies

SOLVING PROBLEM BY HAND	



SCREENSHOTS

CPE010_HOA12.1_SuppleMain.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include "PathCount.h"

int main() {
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost = 25;

    int m = matrix.size();
    int n = matrix[0].size();

    std::cout << "Number of paths with cost " << targetCost << ": " << countPaths(matrix, m - 1, n - 1, targetCost) << std::endl;
    std::cout << std::endl;

    return 0;
}
```

PathCount.h > ...

```h
#ifndef PathCount_h
#define CPathCount_h

#include <vector>

int countPaths(std::vector<std::vector<int>> & costMatrix, int i, int j, int cost) {
    if (cost < 0) {
        return 0;
    }

    if (i == 0 && j == 0) {
        return (cost == costMatrix[0][0]) ? 1 : 0;
    }

    if (i < 0 || j < 0) {
        return 0;
    }

    return countPaths(costMatrix, i - 1, j, cost - costMatrix[i][j]) + countPaths(costMatrix, i, j - 1, cost - costMatrix[i][j]);
}

#endif
```

```
PS C:\DSA\HOA12> & C:\DSA\HOA12\CPE010_HOA12.1_SuppleMain.exe
Number of paths with cost 25: 0
```

OBSERVATION ANALYSIS

- ***First of, this algorithm program falls under recursion algorithm since it counts the number of paths through a cost matrix that add up to a target cost of 25.***
- ***To make a matrix, we used the two-dimensional vector (std::vector<std::vector<int>>) function to store all the elements in 4x4 rows and columns. In the .h file, the if (cost < 0)'s job is to calculate whether the accumulated cost has already reached the target (25).***
- ***The program starts at the bottom-right cell and recursively explores two possible directions: moving up or moving left. Each step is being subtracted from the current cell from the 25 target cost until it reaches the top-left cell.***

CONCLUSION:

Cardenas: **Throughout this Hands-On Activity, I learned about the different algorithmic strategies, including Recursion, Brute Force, Backtracking, Greedy, and Divide-and-Conquer. I understood how each strategy works and how it can be applied to different problems. By learning the importance of choosing an appropriate algorithm because each strategy has its own advantages and limitations in terms of efficiency and problem-solving.**

De Vera: **In this HOA, I manage to categorize and classify our previous HOAs and algorithm techniques according to the strategies they fall under. Although some may be classified on multiple strategies, I chose what I think is closest and suitable based on the codes used. I was able to review the algorithms we had implemented before such as Recursion, Brute Force, Backtracking, Greedy, and Divide-and-Conquer. For example, tree traversals use recursion to visit nodes, while Merge Sort and Quick Sort apply the Divide-and-Conquer strategy by breaking problems into smaller parts. In the supplementary activity, we applied the Recursion method where it needs to explore the matrix table and divide which of the paths can have possible paths that match the target cost. But we arrived to a none possible path.**

.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ***grammar correction and concept clarification***

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.